Beyond the Obstacle

Adaptive Finite Elements and Nested Primal–Dual Active Sets

Victor Sieber
Bachelor's student in Mathematics
Numerical Practicum, University of Zurich

Last edited: November 3, 2026

Abstract

Contents

Abstract **1**

1 The obstacle problem **2**

2 Finite-element discretization **3**
2.1 Meshes and the P1 and P2 bases 3
2.2 Stiffness matrix and constrained quadratic minimization 3

3 The primal–dual active-set method **5**

4 Residual-based error indicator and adaptive refinement **6**
4.1 From jumps to an error residual 6
4.2 Reconstructing the discrete multiplier 7
4.3 From the residual to the implemented indicator 7
4.4 Solve–estimate–mark–refine 8

5 Nested initialization of PDAS **8**

6 Construction of the benchmark problems **9**
6.1 Dirac forces and an exact piecewise-quadratic solution 9
6.2 A smooth manufactured bell with an active obstacle 10

7 Numerical experiments **10**
7.1 Initial solver check: SciPy L-BFGS-B versus PDAS 11
7.2 Smooth benchmark: approximation accuracy 11
7.3 Dirac benchmark: uniform versus adaptive refinement 12
7.4 Efficiency of nested PDAS on identical adaptive meshes 13

8 Conclusion **16**
8.1 Future work 16

8.2 Use of AI 16
8.3 Open questions 16

1 The obstacle problem

We model a string on $(0,1)$, fixed at its endpoints, under a load f. Its displacement u must remain above an obstacle ψ. Set

$$V = H_0^1(0,1), \qquad K = \{v \in V : v \geq \psi \text{ a.e. on } (0,1)\}.$$

Here $H_0^1(0,1)$ consists of functions with a square-integrable weak derivative and zero endpoint values. We choose obstacles with $\psi(0), \psi(1) \leq 0$, so that K is nonempty. Moreover, K is convex: for $v, w \in K$ and $t \in [0,1]$,

$$tv + (1-t)w \;\geq\; t\psi + (1-t)\psi = \psi.$$

The elastic energy and external work give

$$J(v) = \frac{1}{2}\int_0^1 |v'(x)|^2\,dx - \langle f, v\rangle, \qquad u = \operatorname*{arg\,min}_{v\in K} J(v).$$

If f is a function, $\langle f, v\rangle = \int_0^1 fv\,dx$. In one dimension, we may also use point forces: $\langle \delta_\xi, v\rangle = v(\xi)$ for $\xi \in (0,1)$.

Deriving the variational inequality. For any $v \in K$, convexity gives $u + t(v-u) \in K$ when $0 \leq t \leq 1$. Since u minimizes J, the right derivative satisfies

$$0 \leq \frac{d}{dt} J(u + t(v-u))\Big|_{t=0} = \int_0^1 u'(v-u)'\,dx - \langle f, v-u\rangle.$$

Consequently, the weak obstacle problem is to find $u \in K$ such that

$$\boxed{\int_0^1 u'(v-u)'\,dx \geq \langle f, v-u\rangle \qquad \text{for every } v \in K.} \tag{1}$$

We write $a(w,v) = \int_0^1 w'v'\,dx$. The *Poincaré inequality*

$$\|v\|_{L^2(0,1)} \leq \frac{1}{\pi}\|v'\|_{L^2(0,1)} \qquad (v \in H_0^1(0,1))$$

shows that the derivative seminorm controls the full H^1 norm. Hence J is strictly convex on V and the minimizer is unique.

Contact and reaction. At a point where $u > \psi$, small upward and downward variations are allowed, so the weak inequality becomes equality locally: $-u'' = f$. On contact ($u = \psi$), only upward variations remain admissible. Their effect can be represented by a nonnegative *reaction* $\lambda = -u'' - f$. Thus, whenever these quantities are functions, we obtain

$$\underbrace{-u'' = f + \lambda}_{\text{equilibrium}}, \qquad \underbrace{u - \psi \geq 0}_{\text{no penetration}}, \qquad \underbrace{\lambda \geq 0}_{\text{reaction}}, \qquad \underbrace{\lambda(u-\psi) = 0}_{\text{complementarity}}.$$

On the *free region* $u > \psi$, the reaction vanishes; on the *contact region* $u = \psi$, the obstacle may exert a positive reaction. For point forces, the equilibrium equation is understood in the sense of distributions.

2 Finite-element discretization

2.1 Meshes and the P1 and P2 bases

Divide $[0,1]$ into M elements

$$0 = x_0 < x_1 < \cdots < x_M = 1, \qquad T_i = [x_i, x_{i+1}], \quad h_i = x_{i+1} - x_i.$$

Let $V_h \subset V$ contain continuous functions that are polynomial on each element and vanish at the two endpoints.

P_1: one hat function per interior vertex. For each interior node x_i, define the *global* nodal basis function

$$\varphi_i(x) = \begin{cases} (x - x_{i-1})/h_{i-1}, & x \in [x_{i-1}, x_i], \\ (x_{i+1} - x)/h_i, & x \in [x_i, x_{i+1}], \\ 0, & \text{otherwise.} \end{cases}$$

Thus $\varphi_i(x_j) = \delta_{ij}$, and

$$u_h(x) = \sum_{i=1}^{M-1} U_i \varphi_i(x), \qquad U_i = u_h(x_i), \qquad N = M - 1.$$

On an element T_i, only the two neighbouring hats contribute. With $s = (x - x_i)/h_i$,

$$u_h|_{T_i}(x) = U_i(1 - s) + U_{i+1} s.$$

P_2: adding one midpoint to each element. We use the reference interval $\widehat{T} = [-1, 1]$ with nodes $-1, 0, 1$, mapped to T_i by

$$x = \frac{x_i + x_{i+1}}{2} + \frac{h_i}{2} y, \qquad y \in [-1, 1].$$

Construct the quadratic nodal functions as *Lagrange polynomials*: each is one at its own reference node and zero at the other two. For example,

$$\widehat{\varphi}_L(y) = \frac{(y - 0)(y - 1)}{(-1 - 0)(-1 - 1)} = \frac{1}{2} y(y - 1).$$

Analogously,

$$\widehat{\varphi}_M(y) = 1 - y^2, \qquad \widehat{\varphi}_R(y) = \frac{1}{2} y(y + 1).$$

The local quadratic is $u_h|_{T_i} = U_i \widehat{\varphi}_L + U_{m_i} \widehat{\varphi}_M + U_{i+1} \widehat{\varphi}_R$, where $m_i = (x_i + x_{i+1})/2$. There are $M - 1$ interior vertices and M midpoints, hence $N = 2M - 1$ interior degrees of freedom.

Non-aligned mesh example. For $M = 20$ equal elements, $\alpha = 1/3$ lies strictly inside $[0.30, 0.35]$, not at the P_1 vertices or the P_2 midpoint 0.325. Thus a free boundary or derivative jump need not be represented by a node. We will use this feature in the benchmark problems of Section 6.

2.2 Stiffness matrix and constrained quadratic minimization

Writing $u_h = \sum_{j=1}^{N} U_j \varphi_j$ in the energy gives

$$J_h(U) = \frac{1}{2} U^T A U - F^T U, \qquad A_{ij} = \int_0^1 \varphi_i' \varphi_j' \, dx, \quad F_i = \langle f, \varphi_i \rangle.$$

For P_1, the derivative of a hat is $1/h_{i-1}$ on its left element and $-1/h_i$ on its right element. Therefore,

$$A_{ii} = \frac{1}{h_{i-1}} + \frac{1}{h_i}, \qquad A_{i,i-1} = -\frac{1}{h_{i-1}}, \qquad A_{i,i+1} = -\frac{1}{h_i},$$

with all other entries zero. In particular, on a uniform mesh of width h,

$$A_{P_1} = \frac{1}{h}\begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & -1 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & 2 & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix}.$$

For P_2, the chain rule gives $d/dx = (2/h_i)d/dy$, while $dx = (h_i/2)dy$. Therefore,

$$(A_{T_i}^{P_2})_{rs} = \frac{2}{h_i}\int_{-1}^{1} \widehat{\varphi}_r'(y)\widehat{\varphi}_s'(y)\,dy, \qquad r,s \in \{L,M,R\}.$$

As $\widehat{\varphi}_L' = y - \frac{1}{2}$, the (L,L) entry is $(2/h_i)\int_{-1}^{1}(y-\frac{1}{2})^2 dy = 7/(3h_i)$. The neighbouring left–midpoint entry is

$$(A_{T_i}^{P_2})_{LM} = \frac{2}{h_i}\int_{-1}^{1}(y-\tfrac{1}{2})(-2y)\,dy = -\frac{8}{3h_i}.$$

Similarly, the remaining entries give

$$A_{T_i}^{P_2} = \frac{1}{3h_i}\begin{pmatrix} 7 & -8 & 1 \\ -8 & 16 & -8 \\ 1 & -8 & 7 \end{pmatrix}.$$

From local to global (P_2). Each element has three unknown values, ordered as *left vertex, midpoint, right vertex*:

$$\underbrace{(x_i, m_i, x_{i+1})}_{\text{element } T_i} \quad \longleftrightarrow \quad \underbrace{(2i,\ 2i+1,\ 2i+2)}_{\text{global positions}}.$$

Place its 3×3 matrix into these rows and columns of the global matrix $\widetilde{A}$. Where two elements meet, *add* their contributions. For example, the interior vertex x_i belongs to T_{i-1} and T_i, so

$$\widetilde{A}_{2i,2i} = \frac{7}{3h_{i-1}} + \frac{7}{3h_i}.$$

Thus $\widetilde{A} \in \mathbb{R}^{(2M+1)\times(2M+1)}$. Since $u_h(0) = u_h(1) = 0$, remove its first and last rows and columns:

$$\boxed{A \in \mathbb{R}^{(2M-1)\times(2M-1)}}.$$

The same "add at shared nodes" rule was used for the P_1 matrix above.

The same assembly principle applies to P_1. For either degree, A is symmetric and

$$c^T A c = \int_0^1 \left|\left(\sum_{j=1}^{N} c_j\varphi_j\right)'\right|^2 dx > 0 \quad \text{whenever } c \neq 0,$$

since a function in $H_0^1(0,1)$ with zero derivative must be zero. Thus A is positive definite. Let z_i denote the interior P_1 vertices or P_2 vertices/midpoints and set $Z_i = \psi(z_i)$. We solve

$$\boxed{\min_{U\in\mathbb{R}^N,\ U\geq Z} (\tfrac{1}{2}U^T A U - F^T U).} \tag{2}$$

Here $U \geq Z$ is imposed at the degrees of freedom. For P_1 it implies $u_h \geq I_h\psi$; for P_2, nodal feasibility alone need not imply pointwise feasibility between the nodes.

Discrete KKT conditions. Differentiate the discrete energy:

$$\nabla J_h(U) = AU - F =: \Lambda.$$

If $U_i > Z_i$, we can vary U_i slightly in either direction, hence $\Lambda_i = 0$. If $U_i = Z_i$, only a positive variation is admissible, hence $\Lambda_i \geq 0$. Together,

$$\boxed{\Lambda = AU - F, \qquad U - Z \geq 0, \qquad \Lambda \geq 0, \qquad \Lambda_i(U_i - Z_i) = 0 \quad (i = 1, \ldots, N).} \tag{3}$$

The vector Λ is the *dual residual*; it is not yet the nodal values of a multiplier function. In the Dirac example $f = f_{\text{reg}} - 2\delta_\alpha - \delta_\beta$, and the load vector is assembled as

$$F_i = \int_0^1 f_{\text{reg}}\varphi_i \, dx - 2\varphi_i(\alpha) - \varphi_i(\beta).$$

Even when α, β are not nodes, the basis values at those points are well-defined.

3 The primal–dual active-set method

At each degree of freedom, the KKT system (3) says that either $U_i = Z_i$ (*active/contact*) or $\Lambda_i = 0$ (*free*). For any $c > 0$, the four KKT conditions are equivalent to

$$\boxed{\Lambda_i = \max\{0, \Lambda_i - c(U_i - Z_i)\}.}$$

Why? Put $g_i = U_i - Z_i$. If $g_i > 0$, KKT gives $\Lambda_i = 0$, so the right-hand side is $\max(0, -cg_i) = 0$. If $g_i = 0$, KKT gives $\Lambda_i \geq 0$, so the right-hand side is Λ_i. Conversely, the maximum identity implies $\Lambda_i \geq 0$; if $\Lambda_i > 0$ then $g_i = 0$, while if $\Lambda_i = 0$ then $g_i \geq 0$. Thus the identity also implies KKT.

This motivates the test $\Lambda_i - c(U_i - Z_i) > 0$ for an active degree of freedom. We use $c = 1$.

Algorithm 1 Primal–dual active-set (PDAS) method

Input: A, F, Z, parameter $c > 0$, initial contact set $\mathcal{C}^0$

for $k = 0, 1, 2, \ldots$ **do**
  $\mathcal{F}^k \leftarrow \{1, \ldots, N\} \setminus \mathcal{C}^k$ ▷ free indices
  $U_{\mathcal{C}^k} \leftarrow Z_{\mathcal{C}^k}$ ▷ enforce contact
  Solve $A_{\mathcal{F}^k\mathcal{F}^k} U_{\mathcal{F}^k} = F_{\mathcal{F}^k} - A_{\mathcal{F}^k\mathcal{C}^k} Z_{\mathcal{C}^k}$
  $\Lambda \leftarrow AU - F$
  $\mathcal{C}^{k+1} \leftarrow \{i : \Lambda_i - c(U_i - Z_i) > 0\}$
  if $\mathcal{C}^{k+1} = \mathcal{C}^k$ **and** the KKT residual is within tolerance **then**
    return (U, Λ)
  end if
end for

Why is stabilization a stopping criterion? Suppose $\mathcal{C}^{k+1} = \mathcal{C}^k = \mathcal{C}$. For $i \in \mathcal{C}$, the algorithm sets $U_i = Z_i$, and stability gives $\Lambda_i - c(U_i - Z_i) > 0$, hence $\Lambda_i > 0$. For $i \notin \mathcal{C}$, the solved free equations imply $\Lambda_i = (AU - F)_i = 0$. Stability gives $\Lambda_i - c(U_i - Z_i) \leq 0$, hence $U_i - Z_i \geq 0$. In both cases $U_i - Z_i \geq 0$, $\Lambda_i \geq 0$, and $\Lambda_i(U_i - Z_i) = 0$: this is precisely (3). In floating-point arithmetic we also verify KKT numerically (the smooth code uses a 10^{-10} tie tolerance and a 10^{-8} KKT tolerance).

Implementation and independent check. A cold start solves $A\widetilde{U} = F$ and initially selects $\mathcal{C}^0 = \{i : \widetilde{U}_i \leq Z_i\}$. In the Dirac tests we use a tridiagonal P_1 solver and P_2 midpoint elimination; the smooth test uses sparse free-set solves. As an independent check, we also minimized the same quadratic objective with SciPy's `L-BFGS-B`, using lower bounds and gradient $AU - F$ (SciPy reference); the measurements appear in Section 7.

4 Residual-based error indicator and adaptive refinement

4.1 From jumps to an error residual

On a P_1 element T_i, the computed slope is constant. At an interior node, its jump is

$$s_i = u_h'|_{T_i} = \frac{U_{i+1} - U_i}{h_i}, \qquad J_i = s_i - s_{i-1} = u_h'(x_i^+) - u_h'(x_i^-).$$

We set $J_0 = J_M = 0$ because there is no neighbouring element at the boundary. Our first (jump-only) indicator was

$$\eta_{T_i,\text{jump}}^2 = \frac{h_i}{2}(J_i^2 + J_{i+1}^2).$$

It responds to large changes in slope but ignores the force and obstacle reaction. We now motivate the additional volume term.

Step 1: express the error through the weak equation. Let $e = u - u_h$. For this derivation assume a regular load and an exact reaction function λ, so that $a(u, v) = \langle f + \lambda, v\rangle$ for all $v \in V$. By the definition of a,

$$\begin{aligned}\|e'\|_{L^2(0,1)}^2 &= a(e, e) = a(u, e) - a(u_h, e)\\ &= \langle f + \lambda, e\rangle - \int_0^1 u_h' e'\, dx.\end{aligned}$$

The exact λ is unknown. We therefore use a computable reconstruction λ_h (Section 4.2) and add/subtract it:

$$\|e'\|_{L^2}^2 = \underbrace{\langle f + \lambda_h, e\rangle - a(u_h, e)}_{\mathcal{R}_h(e)} + \langle \lambda - \lambda_h, e\rangle.$$

The last term is the reconstruction error; we do not claim it vanishes. We use $\mathcal{R}_h$ to *motivate* a computable refinement indicator.

Step 2: integrate by parts elementwise. For any $v \in V$ and $T_i = [x_i, x_{i+1}]$,

$$\begin{aligned}-\int_{T_i} u_h' v'\, dx &= \int_{T_i} u_h'' v\, dx - [u_h' v]_{x_i}^{x_{i+1}}\\ &= \int_{T_i} u_h'' v\, dx - u_h'(x_{i+1}^-) v(x_{i+1}) + u_h'(x_i^+) v(x_i).\end{aligned}$$

On summing over elements, the endpoint terms vanish because $v(0) = v(1) = 0$. At x_i, the adjacent boundary terms combine into

$$(u_h'(x_i^+) - u_h'(x_i^-))v(x_i) = J_i v(x_i).$$

Thus, writing $(\lambda_h, v) = \int_0^1 \lambda_h v\, dx$ for the regular case,

$$\boxed{\mathcal{R}_h(v) = \sum_{T\in\mathcal{T}_h} \int_T \underbrace{(f + u_h'' + \lambda_h)}_{R_T} v\, dx + \sum_{i=1}^{M-1} J_i v(x_i).}$$

For P_1, $u_h''|_T = 0$; for P_2, it is constant on each element. The jump and volume terms are now both visible.

4.2 Reconstructing the discrete multiplier

The PDAS output $\Lambda = AU - F$ is a vector, whereas R_T requires a function $\lambda_h(x)$. Define the linear functional on V_h by

$$\ell_h(v_h) = a(u_h, v_h) - \langle f, v_h \rangle.$$

For a basis function, $\ell_h(\varphi_i) = (AU - F)_i = \Lambda_i$. Now equip the finite-dimensional space V_h with the L^2 inner product. By the *Riesz representation theorem*, there exists a unique $\lambda_h \in V_h$ such that

$$(\lambda_h, v_h)_{L^2} = \ell_h(v_h) \qquad \text{for every } v_h \in V_h.$$

This is where Riesz enters: it replaces the *functional* ℓ_h by a *function* λ_h representing it in the L^2 inner product. Write $\lambda_h = \sum_{j=1}^{N} \mu_j \varphi_j$ and test with $v_h = \varphi_i$:

$$\sum_{j=1}^{N} \left(\int_0^1 \varphi_j \varphi_i \, dx \right) \mu_j = \Lambda_i.$$

With mass matrix $\mathsf{M}_{ij} = \int_0^1 \varphi_i \varphi_j \, dx$,

$$\boxed{\mathsf{M}\boldsymbol{\mu} = \Lambda.} \tag{4}$$

Importantly, the dual entries Λ_i are *not* point values $\lambda_h(z_i)$; solving this mass-matrix system produces the coefficient vector of the reconstructed function.

4.3 From the residual to the implemented indicator

Step 3: why subtract an interpolant? The discrete equation and the reconstructed multiplier imply

$$\mathcal{R}_h(v_h) = \langle f, v_h \rangle + (\lambda_h, v_h) - a(u_h, v_h) = 0 \qquad \text{for all } v_h \in V_h$$

(for a regular load, up to numerical quadrature). However, the error $e = u - u_h$ is generally *not* in V_h. Choose an averaging interpolant $\mathcal{I}_h e \in V_h$, so that

$$\mathcal{R}_h(e) = \underbrace{\mathcal{R}_h(\mathcal{I}_h e)}_{=0} + \mathcal{R}_h(e - \mathcal{I}_h e) = \mathcal{R}_h(e - \mathcal{I}_h e).$$

Idea: subtract a local average of e; only its variation over each small element remains. This interpolant is used for the mathematical estimate, not by the Python algorithm.

Step 4: where do the factors h_T come from? For an interval ω of length h_ω, write $\bar{e}_\omega = h_\omega^{-1} \int_\omega e(y) \, dy$. Using the fundamental theorem of calculus and Cauchy–Schwarz,

$$\begin{aligned} |e(x) - \bar{e}_\omega| &= \left| \frac{1}{h_\omega} \int_\omega (e(x) - e(y)) \, dy \right| \\ &\le \frac{1}{h_\omega} \int_\omega \left| \int_y^x e'(t) \, dt \right| dy \le h_\omega^{1/2} \|e'\|_{L^2(\omega)}. \end{aligned}$$

Taking the L^2 norm in x gives

$$\|e - \bar{e}_\omega\|_{L^2(\omega)} \le h_\omega \|e'\|_{L^2(\omega)}.$$

This is the one-dimensional *Poincaré inequality with the mean removed*. For a suitable patchwise averaging interpolant, analogous local estimates are

$$\begin{aligned} \|e - \mathcal{I}_h e\|_{L^2(T)} &\lesssim h_T \|e'\|_{L^2(\omega_T)}, \\ |(e - \mathcal{I}_h e)(x_i)| &\lesssim h_T^{1/2} \|e'\|_{L^2(\omega_T)}, \end{aligned}$$

where ω_T consists of T and its neighbours (the constants depend on the local mesh geometry).

Set $w = e - \mathcal{I}_h e$. Apply Cauchy–Schwarz separately to the *volume* and *jump* terms of $\mathcal{R}_h(w)$:

$$\left|\int_T R_T w\,dx\right| \le \|R_T\|_{L^2(T)}\|w\|_{L^2(T)} \lesssim \underbrace{h_T\|R_T\|_{L^2(T)}}_{\text{volume factor}}\|e'\|_{L^2(\omega_T)},$$

$$|J_i w(x_i)| \lesssim \underbrace{h_T^{1/2}|J_i|}_{\text{jump factor}}\ \|e'\|_{L^2(\omega_T)}.$$

Square these two residual factors. Each interior jump belongs to two neighbouring elements, so assign half to each. This motivates our *implemented* indicator:

$$\boxed{\eta_T^2 = h_T^2\|f_{\mathrm{reg}} + u_h'' + \lambda_h\|_{L^2(T)}^2 + \frac{h_T}{2}(J_{T,L}^2 + J_{T,R}^2), \qquad \eta_h = \Big(\sum_{T\in\mathcal{T}_h}\eta_T^2\Big)^{1/2}.} \tag{5}$$

The volume integral uses four-point Gauss–Legendre quadrature. *Caveat:* the exact–reconstructed multiplier difference in Step 1 has not been bounded here. In the Dirac benchmark the point loads enter FEM assembly but are not added explicitly to the indicator. Hence this is a motivated refinement indicator, *not* a general reliability theorem.

4.4 Solve–estimate–mark–refine

With maximum marking parameter $\theta = 0.5$, mark the elements satisfying

$$\eta_T \ge \theta \max_{S\in\mathcal{T}_h}\eta_S,$$

then bisect every marked interval at its midpoint. Repeat

$$\boxed{\text{solve} \longrightarrow \text{estimate} \longrightarrow \text{mark} \longrightarrow \text{refine}}$$

until $\eta_h \le \text{tol}$. This is the maximum-marking rule used in all adaptive experiments.

5 Nested initialization of PDAS

After refinement, the previous mesh solution is already a useful approximation to the new one. Instead of recomputing a cold initial guess, we transfer the coarse primal and dual information to the fine mesh.

Primal initialization. Let H denote the coarse mesh and h the refined mesh. Evaluate the old FE function u_H at the new fine-grid nodes:

$$U_h^0 = I_h U_H.$$

Here I_h is the nodal interpolation operator; coarse vertex values are retained.

Dual initialization. The coarse dual vector $\Lambda_H = A_H U_H - F_H$ consists of *functional coefficients*, so simply interpolating its entries as if they were function values is not appropriate. We instead use the same L^2-Riesz reconstruction as in (4):

$$\underbrace{\mathsf{M}_H\boldsymbol{\mu}_H = \Lambda_H}_{\text{represent the coarse dual as a function}} \longrightarrow \underbrace{\boldsymbol{\mu}_h^0 = I_h\boldsymbol{\mu}_H}_{\text{interpolate function values}} \longrightarrow \underbrace{\Lambda_h^0 = \mathsf{M}_h\boldsymbol{\mu}_h^0}_{\text{recover fine dual}}.$$

Combining both initializations gives the main transfer formula

$$\boxed{U_h^0 = I_h U_H, \qquad \Lambda_h^0 = \mathsf{M}_h I_h \mathsf{M}_H^{-1}\Lambda_H.} \tag{6}$$

These initial guesses determine the fine contact set

$$\mathcal{C}_h^0 = \{i : (\Lambda_h^0)_i - c((U_h^0)_i - (Z_h)_i) > 0\}.$$

The subsequent PDAS steps are identical to the cold method: only the *initial contact set* changes.

Algorithm 2 Adaptive FEM with L^2-nested PDAS initialization

Input: Initial mesh $\mathcal{T}_0$, estimator tolerance ε, $\theta = 0.5$

Solve on $\mathcal{T}_0$ by cold PDAS (Algorithm 1)
for adaptive levels $\ell = 0, 1, 2, \ldots$ **do**
  Compute λ_h and the indicators η_T from (5)
  if $\eta_h \leq \varepsilon$ **then**
    return current mesh and u_h
  end if
  Mark by the maximum rule; bisect the marked elements
  Assemble $A_h, F_h, Z_h, \mathsf{M}_h$ on the refined mesh
  Transfer (U_H, Λ_H) by (6) and form $\mathcal{C}_h^0$
  Run PDAS (Algorithm 1) from $\mathcal{C}_h^0$
end for

For a controlled comparison, cold PDAS also solves the problem on each *identical* adaptive mesh. We check that both results agree, so initialization affects iterations and timing but not approximation accuracy.

6 Construction of the benchmark problems

6.1 Dirac forces and an exact piecewise-quadratic solution

Choose the constant obstacle $\psi = -1$ and the force

$$f = -6 - 2\delta_\alpha - \delta_\beta, \qquad \alpha = \tfrac{1}{3}, \quad \beta = \tfrac{2}{3}.$$

Away from the point loads, the free-region equation $-u'' = -6$ implies $u'' = 6$, so the solution is quadratic on each free interval. We seek a contact interval $[\alpha, \gamma]$ where $u = -1$, so $u' = 0$ on contact.

Left free region. Write $u(x) = 3x^2 + ax + b$. The endpoint condition $u(0) = 0$ gives $b = 0$, and continuity at α gives $u(\alpha) = -1$, hence $a = -4$. Therefore $u(x) = 3x^2 - 4x$ for $x \leq \alpha$. Its derivative changes from -2 to 0 at α, so $[u']_\alpha = 2$, corresponding to the force $-2\delta_\alpha$.

Right free region. Leaving contact smoothly at γ gives $u(x) = -1 + 3(x - \gamma)^2$ until β. A force $-\delta_\beta$ requires $[u']_\beta = 1$. Write $d = \beta - \gamma$; at β we have $u(\beta) = -1 + 3d^2$, $u'(\beta^-) = 6d$, $u'(\beta^+) = 6d + 1$. Continuing quadratically to $x = 1$ and imposing $u(1) = 0$ yields

$$0 = (-1 + 3d^2) + (6d + 1)\frac{1}{3} + 3\left(\frac{1}{3}\right)^2 = 3d^2 + 2d - \frac{1}{3}.$$

The positive root is $d = (\sqrt{2} - 1)/3$, hence $\gamma = \beta - d = 1 - \sqrt{2}/3$.

Together these conditions determine the exact solution

$$u(x) = \begin{cases} 3x^2 - 4x, & 0 \leq x \leq \alpha, \\ -1, & \alpha < x \leq \gamma, \\ -1 + 3(x - \gamma)^2, & \gamma < x \leq \beta, \\ 3x^2 + (2\sqrt{2} - 5)x + 2 - 2\sqrt{2}, & \beta < x \leq 1. \end{cases}$$

The reaction is $\lambda = 6\mathbf{1}_{(\alpha,\gamma)}$ (a.e.), since $u'' = 0$ on contact and $u'' = 6$ elsewhere. At the point forces,

$$[u']_\alpha = 2, \qquad [u']_\gamma = 0, \qquad [u']_\beta = 1.$$

Thus $-u'' = f + \lambda$ in distributions. The solution is continuous and piecewise quadratic but its derivative has jumps: a particularly informative test for localized refinement, especially with P_2 elements.

6.2 A smooth manufactured bell with an active obstacle

For a second test, we want *no Dirac forces*, a non-polynomial exact solution, nonconstant obstacle, and nonzero reaction. We construct the example backwards: choose the solution and contact region first, then build the obstacle, multiplier and load to satisfy the KKT conditions.

1. Prescribe the solution. Choose a negative bell with zero endpoint values,

$$u(x) = -\sin(\pi x)e^{-6(x-1/2)^2}, \qquad u(0) = u(1) = 0.$$

2. Construct the obstacle. Prescribe contact on $[1/3, 2/3]$ using

$$d(x) = \max\{0, 1-3x, 3x-2\}, \qquad \psi(x) = u(x) - d(x)^6.$$

Here $d(x) = 0$ exactly on $[1/3, 2/3]$ and $d(x) > 0$ outside. Thus $u - \psi = d^6 \geq 0$, with the desired contact set. The factor 3 makes $d(0) = d(1) = 1$, so $\psi(0) = \psi(1) = -1$; the sixth power makes the transitions C^5.

3. Prescribe the reaction. We need $\lambda \geq 0$ only where $u = \psi$. Set

$$\lambda(x) = \left[36\left(x - \tfrac{1}{3}\right)\left(\tfrac{2}{3} - x\right)\right]_+^6, \qquad [t]_+ = \max\{0, t\}.$$

The factor 36 normalizes the peak to $\lambda(1/2) = 1$. Its support lies inside the contact interval, hence $\lambda(u - \psi) = 0$.

4. Determine the forcing. Impose equilibrium $-u'' = f + \lambda$, giving $f = -u'' - \lambda$. With $t = x - 1/2$,

$$u''(x) = e^{-6t^2}\left[(12 + \pi^2 - 144t^2)\sin(\pi x) + 24\pi t\cos(\pi x)\right].$$

This verifies the continuous KKT conditions by construction. The exact u is C^∞ and non-polynomial; ψ, λ and f are C^5. In contrast with the Dirac benchmark, a P_2 element cannot reproduce the smooth exact solution exactly.

7 Numerical experiments

For both benchmarks, report the H^1-seminorm error and L^2 error,

$$E_{H^1} = \left(\sum_{T\in\mathcal{T}_h}\int_T |u' - u_h'|^2 dx\right)^{1/2}, \qquad E_{L^2} = \left(\sum_{T\in\mathcal{T}_h}\int_T |u - u_h|^2 dx\right)^{1/2}.$$

We evaluate errors with 12-point elementwise Gauss–Legendre quadrature, splitting elements at the exact transition points. The smooth load uses 10-point quadrature; the indicator uses four-point quadrature. Error evaluation does not affect the computed meshes.

7.1 Initial solver check: SciPy L-BFGS-B versus PDAS

Before the adaptive studies, we compared PDAS with the L-BFGS-B method implemented in SciPy on a P_1 example with $f = -1$, $\psi(x) = x(1-x) - \frac{3}{2}\alpha^2$ and $\alpha = 0.2$. Both solve the same constrained minimization problem (2). Times are medians of seven *wall-clock* runs per mesh (`perf_counter`), unlike the later process-CPU timings.

Elements	DOFs	L-BFGS-B iter.	PDAS iter.	L-BFGS-B [ms]	PDAS [ms]
20	19	12	4	0.266	0.074
80	79	35	13	0.901	0.376
320	319	169	51	9.322	7.534

Table 1: Selected measurements from `experiments/benchmark_solvers.py`. At 320 elements the discrete solutions differ by 8.82×10^{-8}; this is an independent solver check, not a broad speed comparison.

7.2 Smooth benchmark: approximation accuracy

For the problem in Section 6.2, uniform P_1 converges approximately as N^{-1} in H^1 and N^{-2} in L^2; uniform P_2 gives N^{-2} and N^{-3}. The adaptive curves have similar slopes, while the reconstructed multiplier agrees closely with the exact reaction.

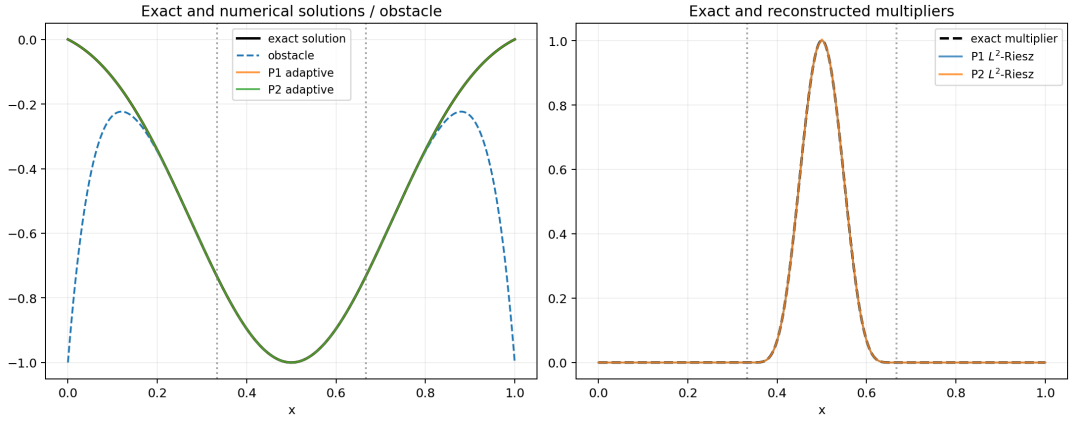


Figure 1: Smooth benchmark (Section 6.2): exact and FE solutions, obstacle, and exact versus reconstructed multiplier.

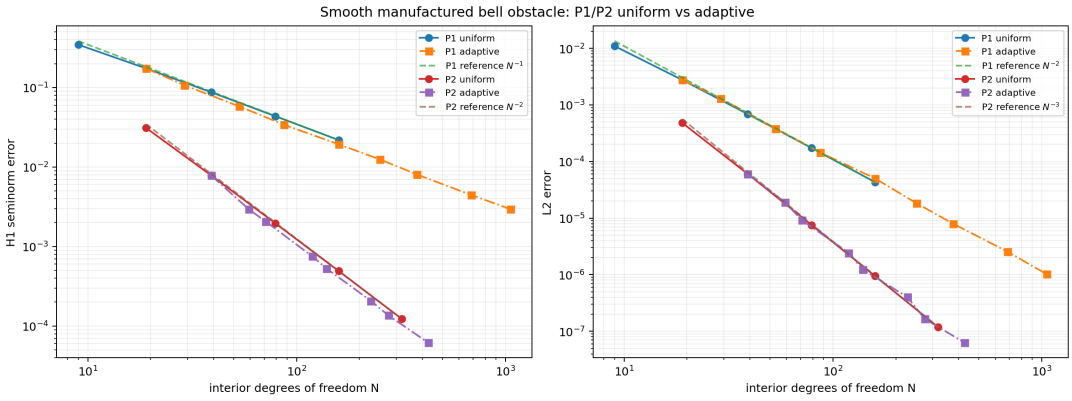


Figure 2: Smooth P_1/P_2 errors versus DOFs: uniform and adaptive refinement with reference slopes.

7.3 Dirac benchmark: uniform versus adaptive refinement

For the singular problem in Section 6.1, uniform P1 and P2 produce observed rates near $N^{-1/2}$ in H^1 and $N^{-3/2}$ in L^2. Local refinement reduces the errors much faster for the piecewise-quadratic P_2 solution. These are benchmark-specific observations; unusually steep adaptive slopes are not claimed as general orders.

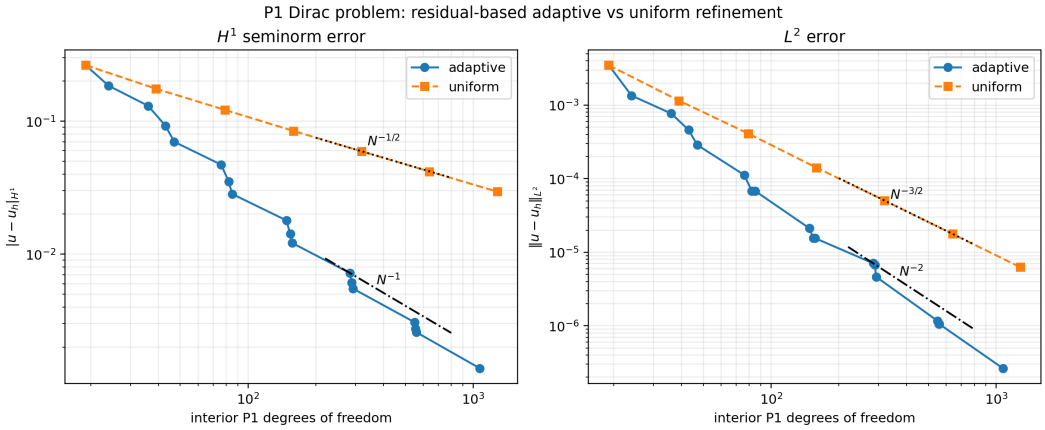


Figure 3: Dirac P_1: uniform and residual-adaptive errors, computed by elementwise Gaussian quadrature.

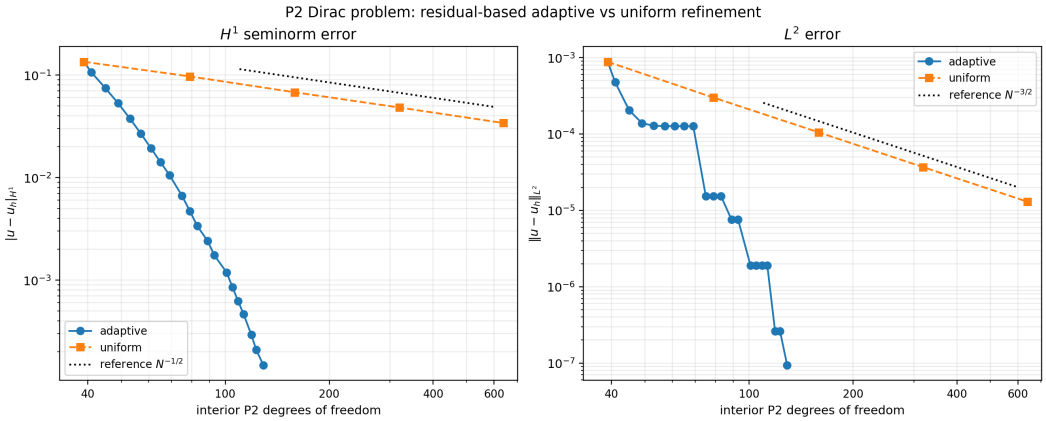


Figure 4: Dirac P_2: adaptive refinement concentrates DOFs near the derivative jumps and transition points.

7.4 Efficiency of nested PDAS on identical adaptive meshes

Now compare *cold-start* and L^2*-nested* PDAS on the same mesh at each adaptive level (a level is one mesh, before the next refinement). For each method and each level, the solver is timed five times with `process_time()`; one plotted CPU point is the *median of those five timings*, not a median across mesh levels. The coarse initial solve is charged to both methods.

For each benchmark, we also report the ratio

$$\text{solver-stage speedup} = \frac{\sum_{\ell=0}^{L} \text{median}_{r=1,\dots,5}\, t_{\ell,r}^{\text{cold}}}{\sum_{\ell=0}^{L} \text{median}_{r=1,\dots,5}\, t_{\ell,r}^{\text{nested}}}.$$

Here $L+1$ is the number of mesh levels, N is the number of interior FE degrees of freedom, and the "final iterations" refer to the *last* adaptive mesh. Nested timings include primal/dual transfer and PDAS; neither method's timing includes assembly, the indicator, marking, refinement or error quadrature. The methods agree on the discrete solution (within 10^{-8}), so accuracy curves are common to both.

Case	FEM	Levels	Final N	Final iter. cold/nested	Cold [s]	Nested [s]	Speedup
Smooth	P_1	9	1055	90/2	0.05682	0.00688	8.3×
Smooth	P_2	8	427	86/15	0.05336	0.01084	4.9×
Dirac	P_1	18	1071	245/5	0.54651	0.01993	27.4×
Dirac	P_2	21	129	51/4	0.05620	0.00644	8.7×

Table 2: Sum of per-mesh median solver-stage CPU times (five repeats per method and mesh), with the final-mesh PDAS iteration counts.

In both test problems, nested initialization reduces the iteration count and measured solver cost. These speedups compare the two initializations *within* each benchmark and polynomial degree; they are not end-to-end runtime gains or claims about different solver implementations. Figures 5 and 6 show the full mesh histories.

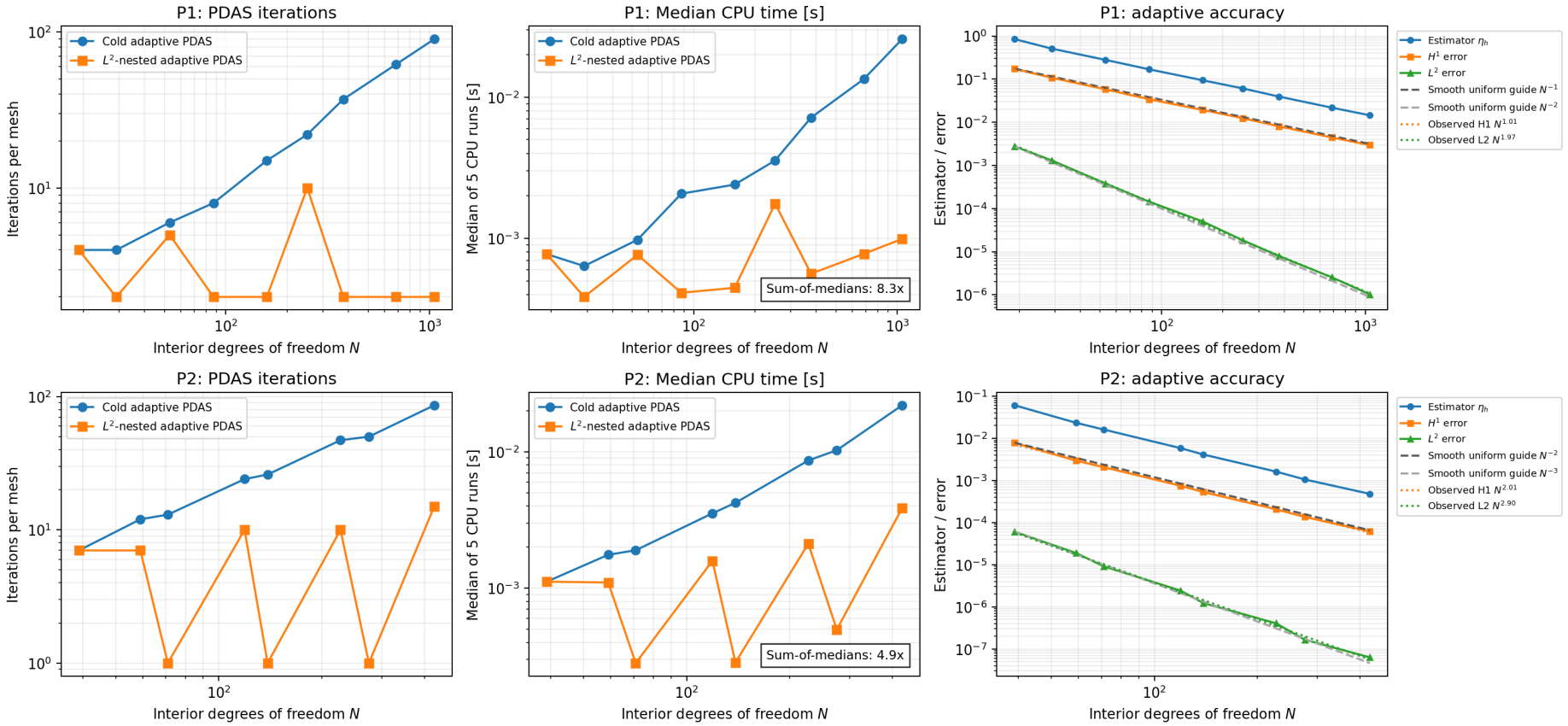


Figure 5: Smooth benchmark (Section 6.2): cold versus L^2-nested PDAS on identical adaptive meshes for P_1 and P_2.

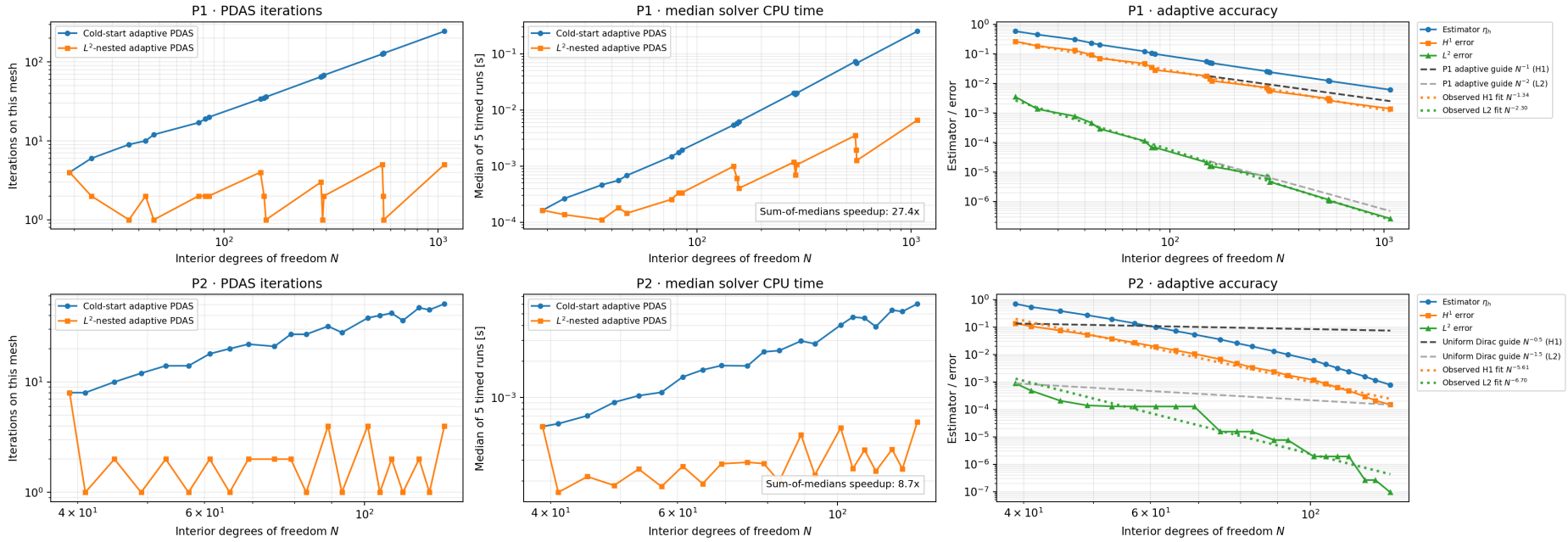


Figure 6: Dirac benchmark (Section 6.1): cold versus L^2-nested PDAS on identical adaptive meshes for P_1 and P_2. Dashed reference guides and dotted finite-range fitted slopes have different meanings.

8 Conclusion

8.1 Future work

8.2 Use of AI

8.3 Open questions